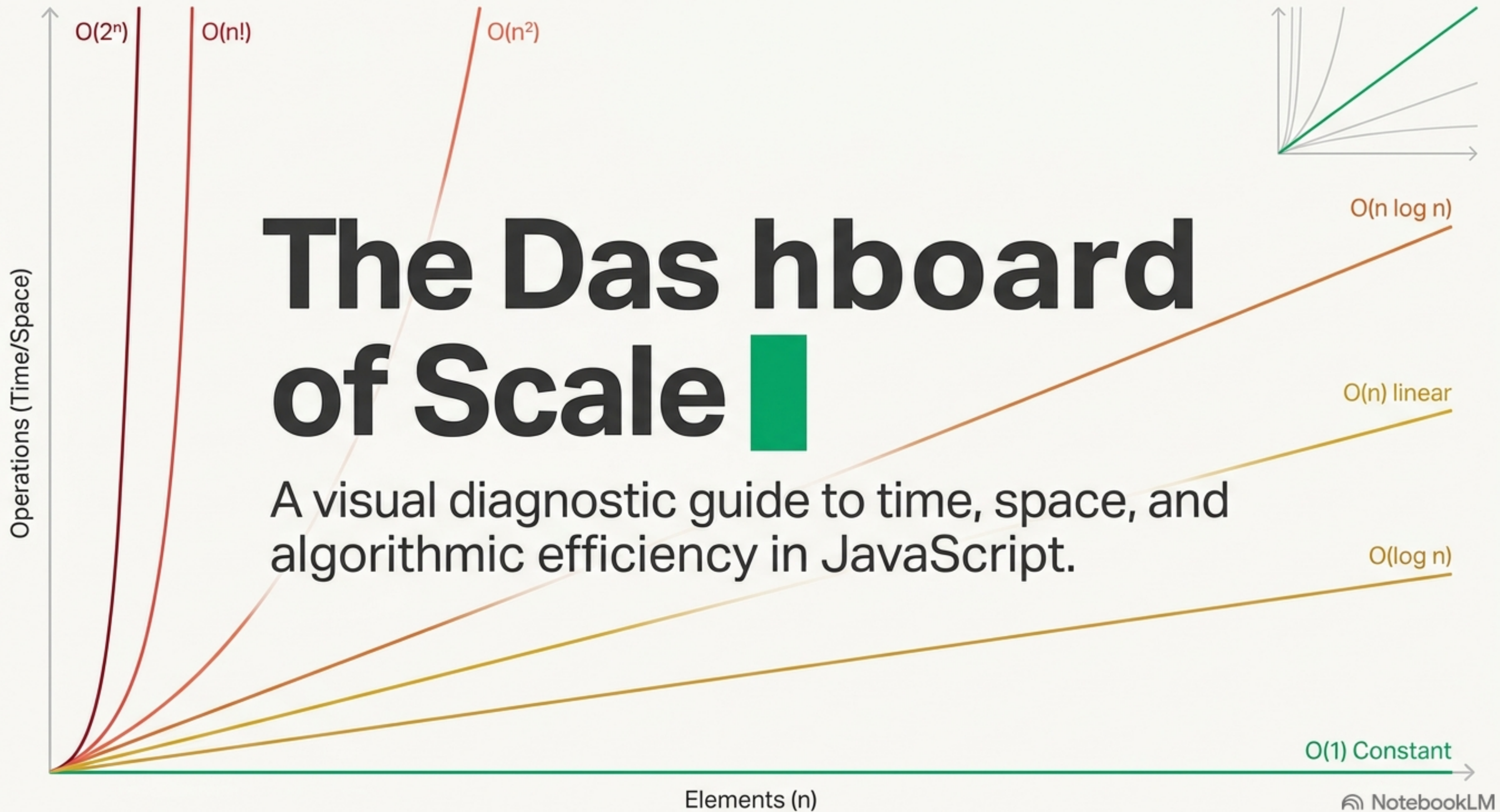




The Wrong Way



Execution Time

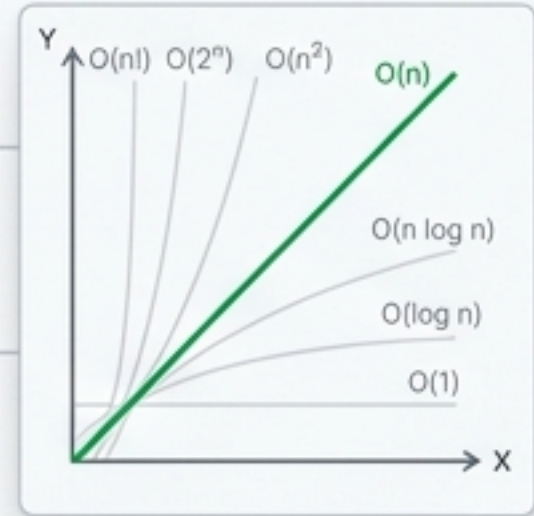
Measuring in milliseconds is unreliable. It depends entirely on hardware strength, CPU load, and background processes.

The Right Way



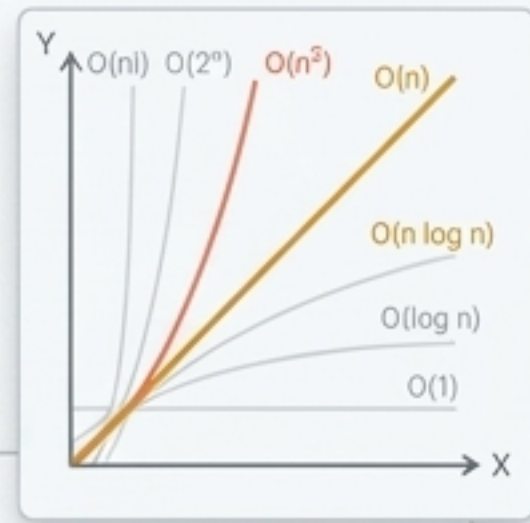
Operations

Big O measures the **worst-case scenario**: how the total number of **operations** scales as the input size (n) approaches infinity.



An algorithm might run instantly with 10 items, but crash the system with 1,000. Big O reveals the future of your function.

The 4-Step Diagnostic Flowchart



Step 1



Assume the Worst

Always calculate for the worst-case scenario (e.g., the target item is at the very end of the array).

Step 2



Assign Unique Variables

Different inputs need different variables. Looping through two separate arrays is $O(a + b)$, not $O(n)$.

Step 3



Drop the Constants

$O(2n)$ or $O(n / 2)$ simplifies to $O(n)$.
Scaling patterns ignore flat multipliers.

Step 4



Drop Non-Dominant Terms

In $O(n^2 + n + 1)$, the quadratic term completely dominates as inputs grow. Simplify to $O(n^2)$.

Perfect Scalability

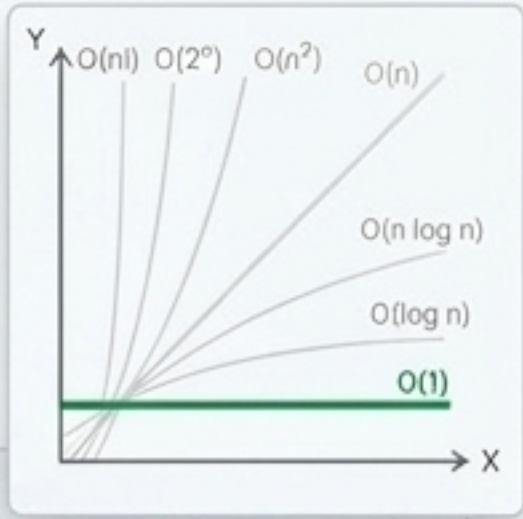
$O(1)$ Constant Time



No matter how massive the input data gets, the execution takes the exact same amount of time. One operation is one operation.

```
function timesTwo(num) {  
  return 2 * num;  
}
```

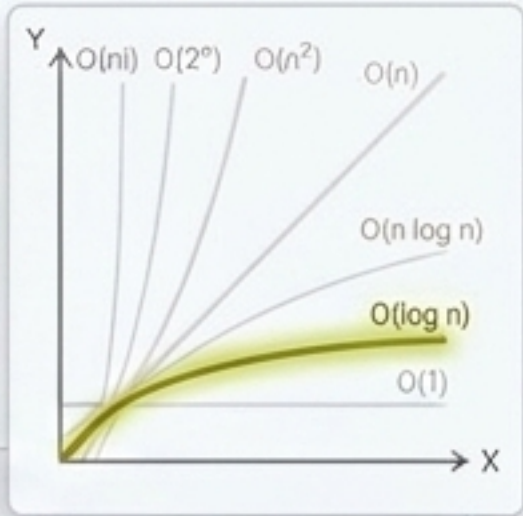
$2 * 5$ takes the exact same computational effort as $2 * 2,000,000$.



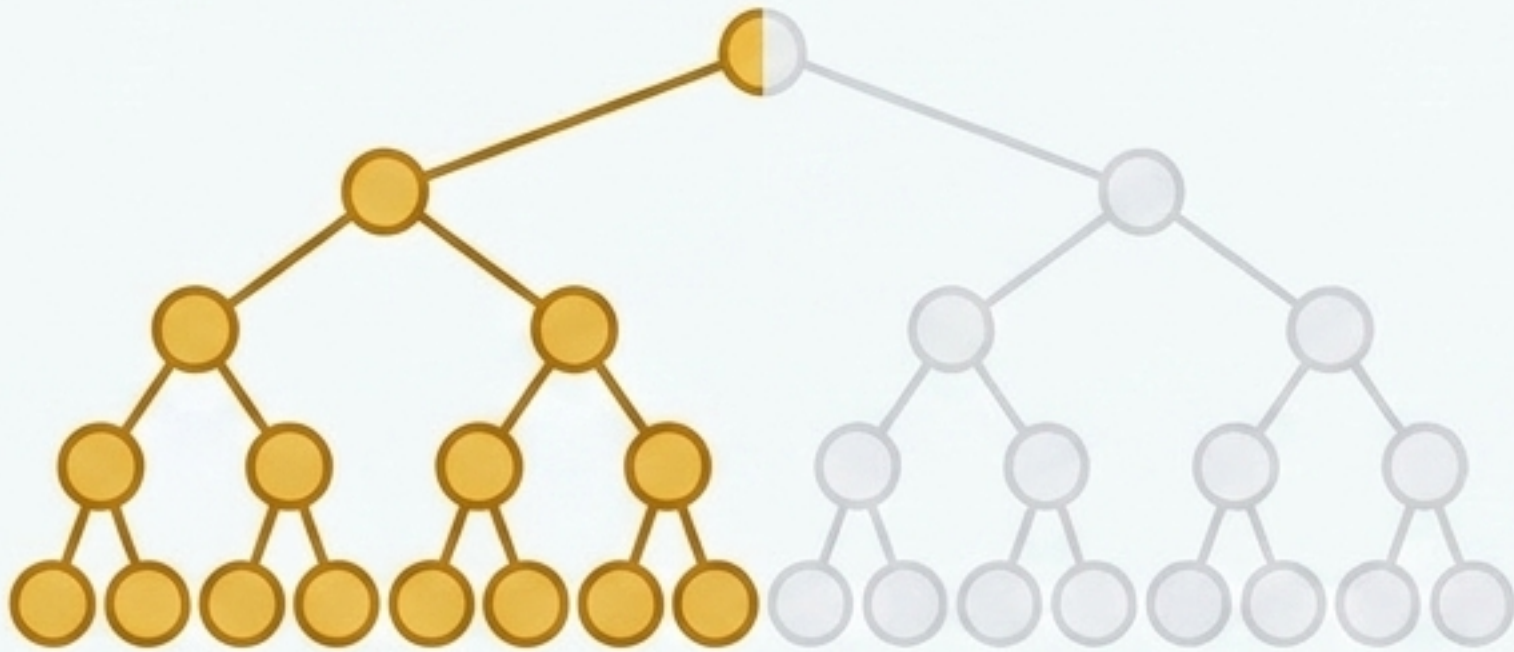
Built-in Methods: `Array.push()` and `Array.pop()` are $O(1)$ because they simply add or remove the final index without rearranging any existing data.

Divide and Conquer

$O(\log n)$ Logarithmic Time



The Elimination Tree



The dataset is cut in half on each step.
To increase operations by just 1, the
total input size must double.

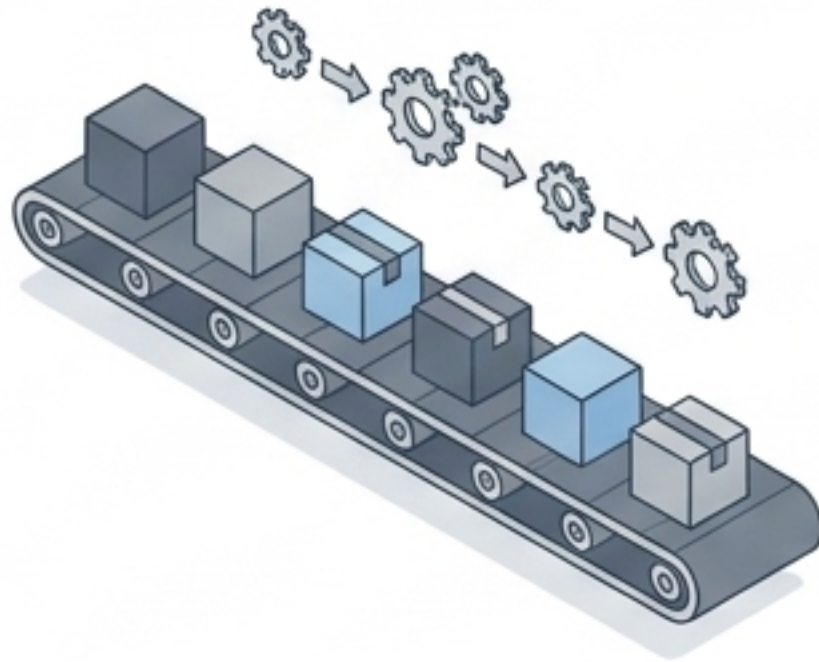
```
for (let i = 1; i < n; i *= 2) {  
  // Process half the remaining data  
}
```

This mathematical halving or doubling
is the absolute fingerprint of
Logarithmic time.

The second-best outcome. Typical for Binary Search algorithms operating on sorted data.

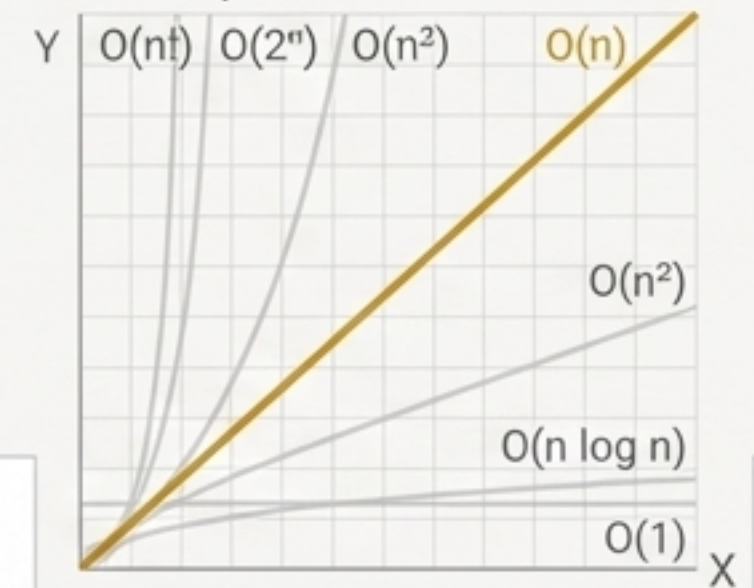
The 1-to-1 Ratio

$O(n)$ Linear Time



Operations scale directly and proportionally with the input size. 10 items = 10 operations. 1,000 items = 1,000 operations.

Mini-Map



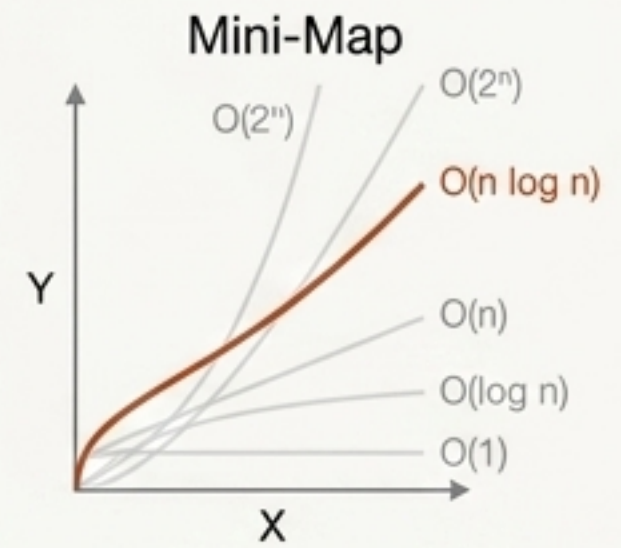
```
function reverseArray(arr) {  
  let reversed = [];  
  for (let i = arr.length - 1; i >= 0; i--) {  
    reversed.push(arr[i]);  
  }  
  return reversed;  
}
```

Loops exactly 'n' times.

⚠ WARNING: Beware `Array.unshift()` and `Array.shift()`. Adding or removing from the front of an array forces JavaScript to **re-index** every single subsequent item. It is secretly $O(n)$ under the hood.

The Worst of the Best

$O(n \log n)$ Linearithmic Time



$O(n)$ [Linear Sequential Loop]

×

$O(\log n)$ [Halving Inner Process]

$O(n \log n)$

Usually found in advanced sorting algorithms like Merge Sort or Quicksort. It pairs an outer sequential loop with an inner divide-and-conquer mechanism.

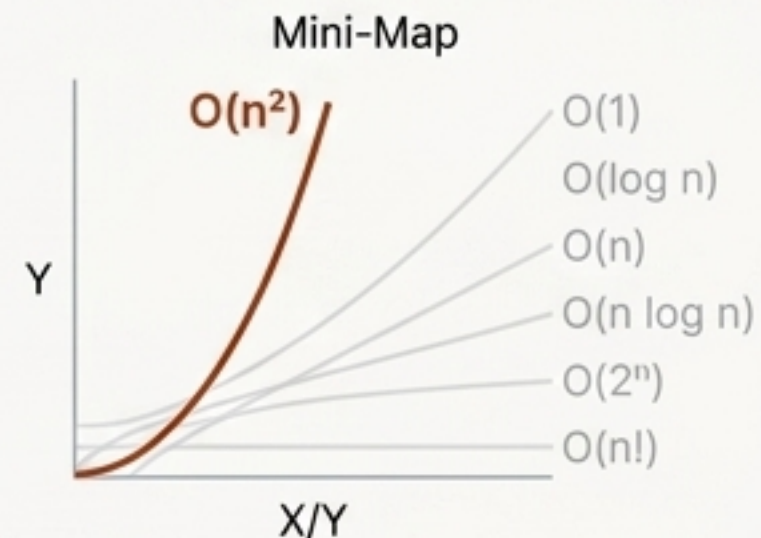
```
for (let i = 0; i < n; i++) {  
  for (let j = 1; j < n; j *= 2) {  
    // Operations  
  }  
}
```

$O(n)$

$O(\log n)$

The Nested Loop Trap

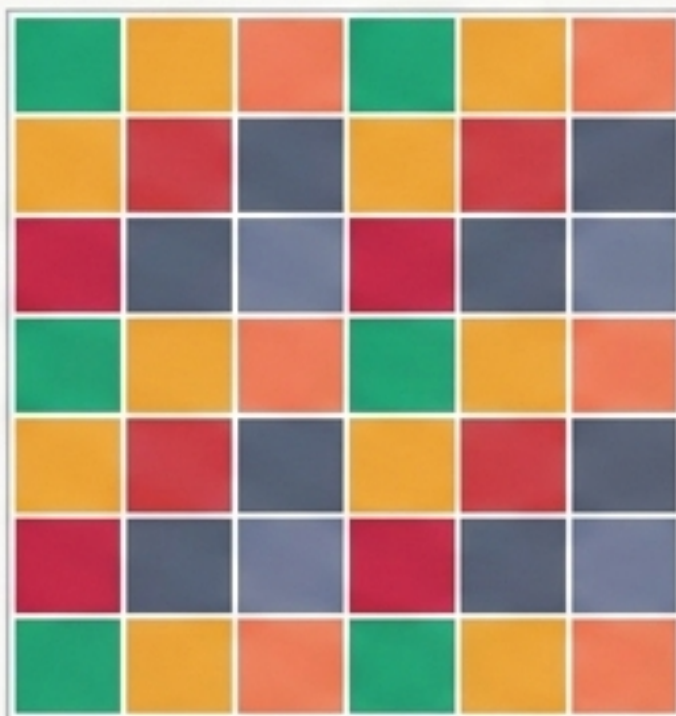
$O(n^2)$ Quadratic Time



3x3 = 9 Operations



8x8 = 64 Operations



```
function multiplyAll(arr) {  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = 0; j < arr.length; j++) {  
      console.log(arr[i] * arr[j]);  
    }  
  }  
}
```

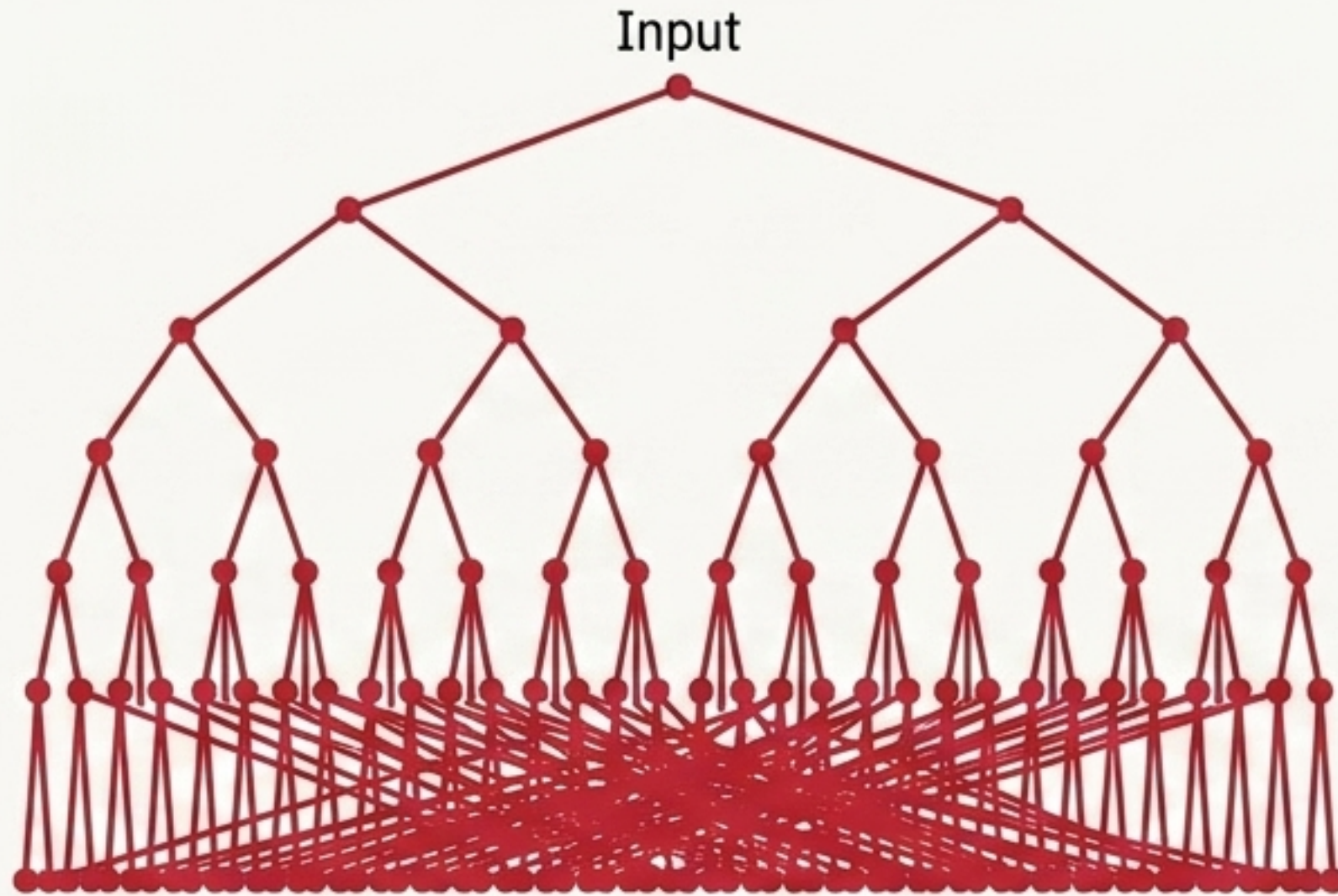
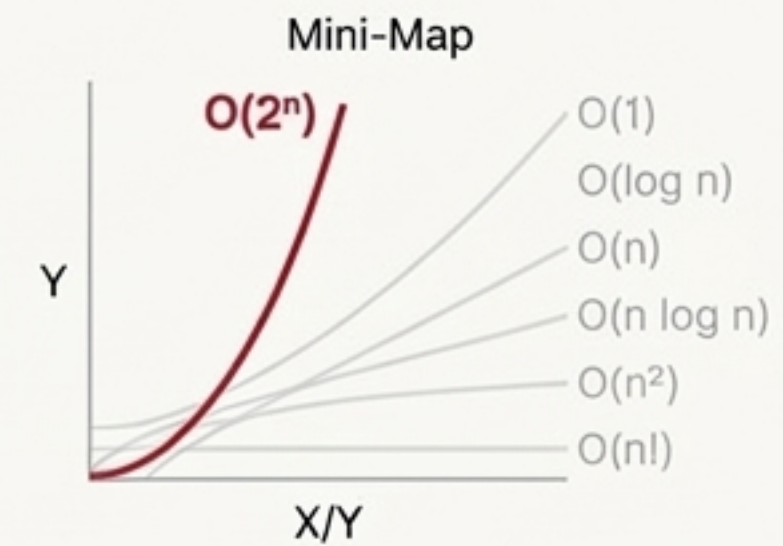
Nested iteration
over the same
dataset
guarantees $O(n^2)$.

Operations grow at an accelerating rate.
Doubling the input roughly quadruples the
total computational work.

Diagnostic Rule: Two separate sequential loops are $O(a + b)$ [Linear].
One nested loop is $O(n^2)$ [Quadratic]. Avoid nested loops for large datasets.

The Branching Explosion

$O(2^n)$ Exponential Time



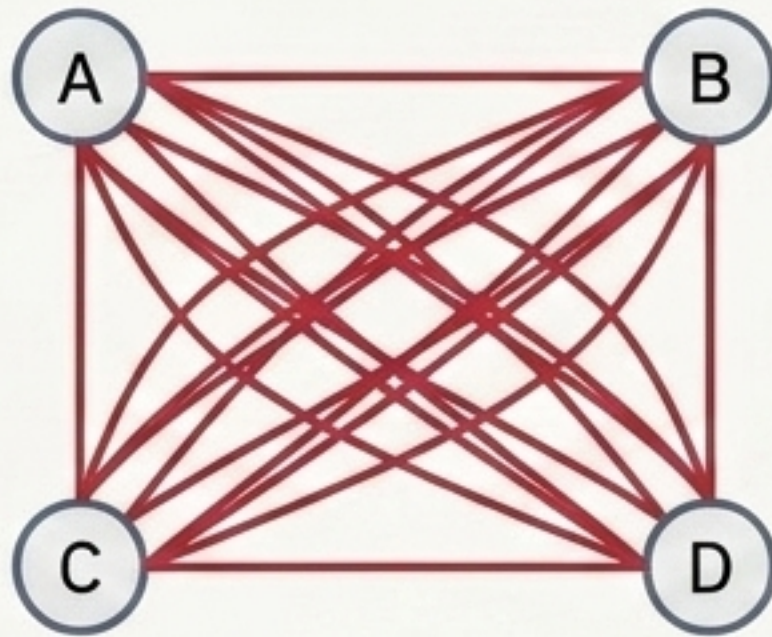
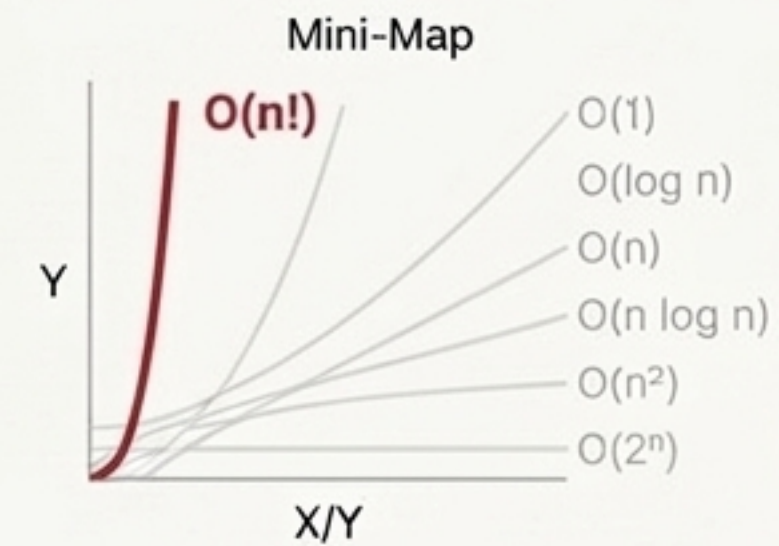
```
function fibonacci(num) {  
  if (num <= 1) return num;  
  return fibonacci(num - 1) + fibonacci(num - 2);  
}
```

Recursive calls that spawn multiple new calls per execution guarantee an exponential meltdown.

The exact opposite of Logarithmic time. Every time you add just ONE item to the input, the total number of operations doubles.

Total System Collapse

$O(n!)$ Factorial Time



3 cities = 6 routes.

4 cities = 24 routes.

14 cities = over 87 billion routes. A browser will completely freeze trying to calculate $n=14$.

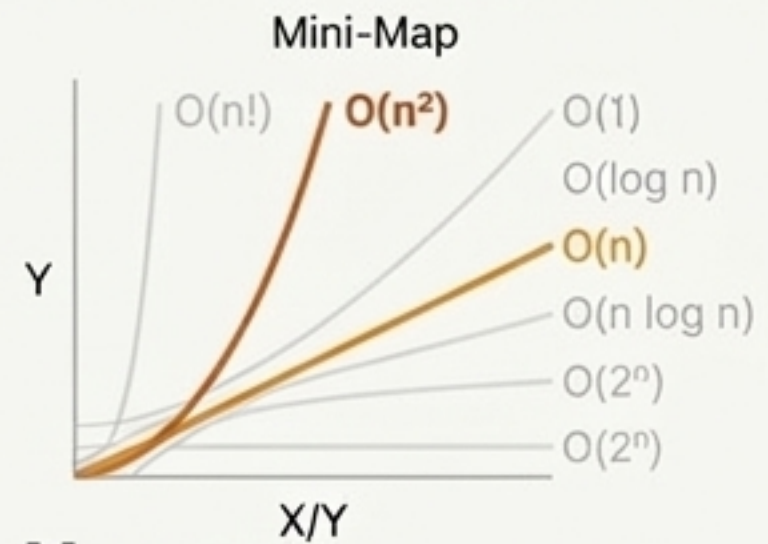
```
JavaScript

function getPermutations(arr) {
  if (arr.length === 0) return [[]];
  let result = [];
  for (let i = 0; i < arr.length; i++) {
    let rest = getPermutations(arr.slice(0, i).concat(arr.slice(i+1)));
    // Add logic
  }
  return result;
}
```

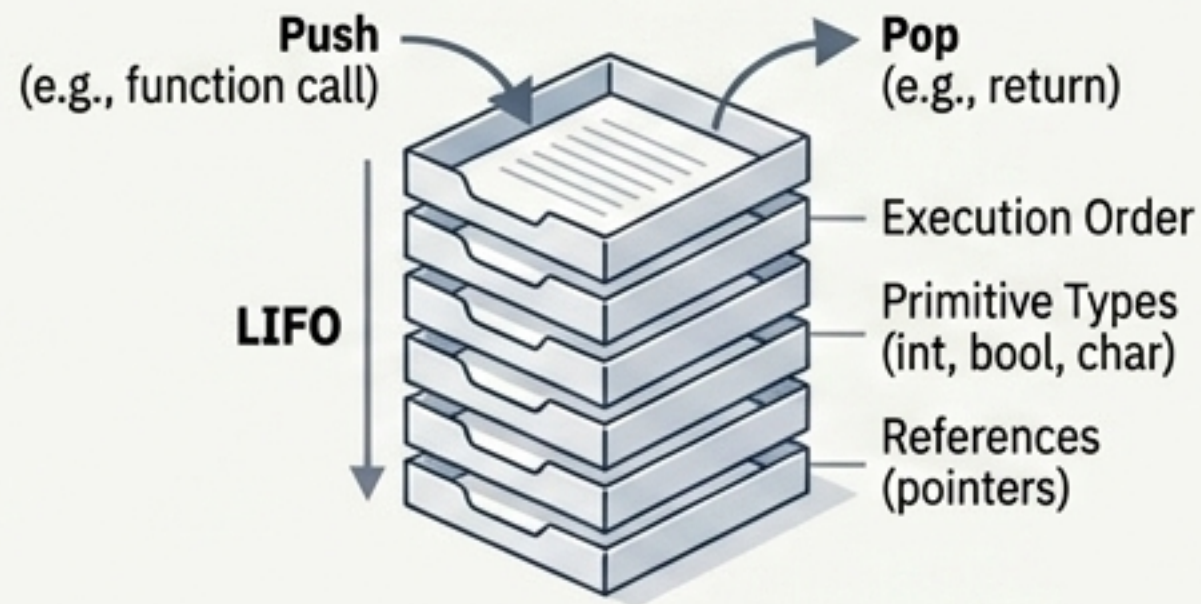
AVOID AT ALL COSTS. Factorial growth means adding one input multiplies the operations by the new total size.

Anatomy of Space Complexity

Time is about speed. Space is about memory.



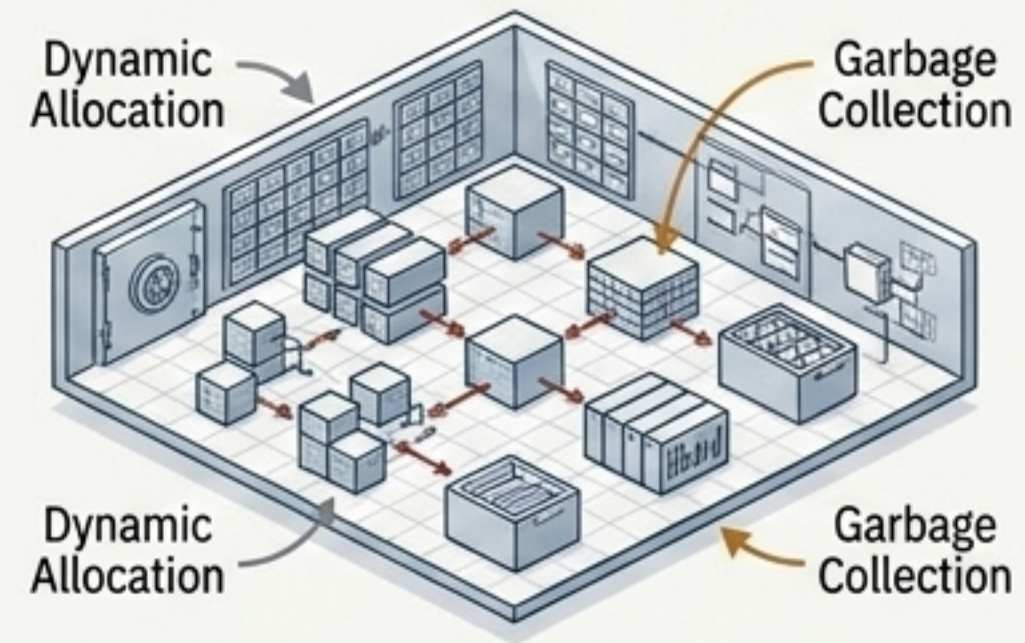
The Call Stack



Operates First In, Last Out (**FILO**). Stores execution order, primitive types, and references.

Too many recursive function calls will exceed this hard limit, causing the infamous “Stack Overflow”.

The Memory Heap



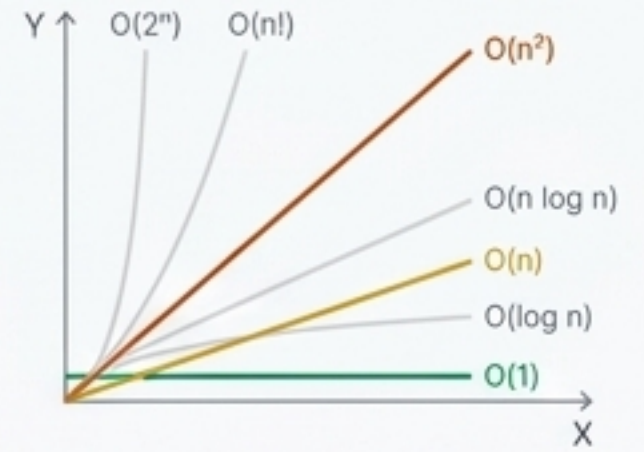
Stores dynamic, non-primitive types (Objects, Arrays, Functions).

Creating new arrays or object maps inside a function based on input length drastically adds to Auxiliary Space.

To diagnose Space Complexity, ask: “Does this function create NEW memory allocations (new variables, objects, array clones) that scale with the input?”

The Ultimate Trade-off

The overarching insight of software engineering:
You can often buy speed by spending memory.



Brute Force Approach

```
for (let i = 0; i < nums.length; i++) {  
  for (let j = i + 1; j < nums.length; j++) {  
    // Check if sum equals target  
  }  
}
```

Time: $O(n^2)$ [Quadratic]

Space: $O(1)$ [Constant]

No extra memory is used, but searching takes exponential time due to nested loops.

Hash Map Approach

```
let map = {};  
for (let i = 0; i < nums.length; i++) {  
  // Store values in map for instant lookup  
}
```

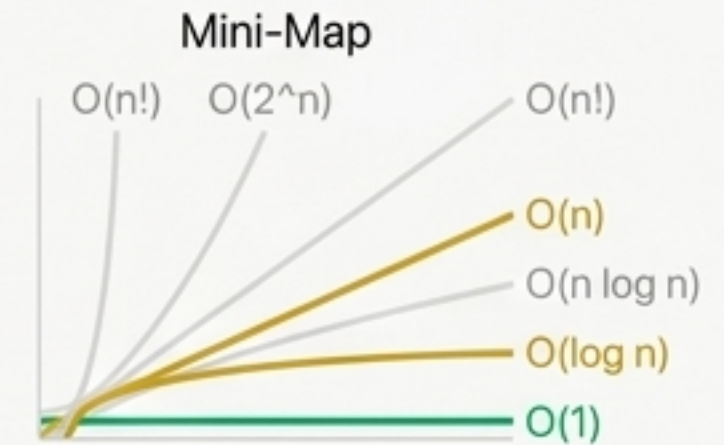
Time: $O(n)$ [Linear]

Space: $O(n)$ [Linear]

By temporarily sacrificing memory to store values in a Hash Map, we eliminate the nested loop, transforming a million iterations into a thousand.

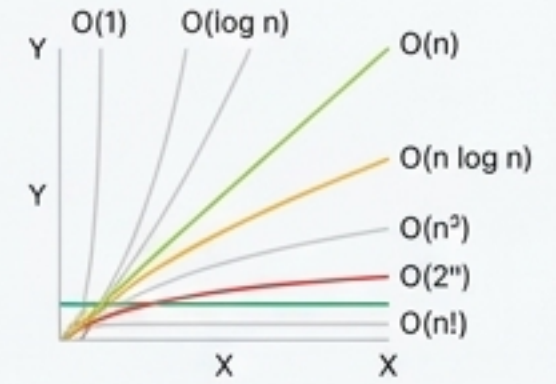
Built-in Diagnostics

Time Complexities for Standard JavaScript Data Structures



Data Structure	Access	Search	Insertion	Deletion
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
(Insertion/Deletion are $O(n)$ due to re-indexing elements)				
Hash Table (JS Object)	$O(1)$	$O(1)$	$O(1)$	$O(1)$
(Average case for key-value pair operations)				
Binary Search Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
(Assumes a balanced tree structure)				

The Master Big-O Cheat Sheet



Complexity Notation	Name	Simple Metaphor	JS Example Trigger
$O(1)$	Constant	1 operation	<code>Array.push()</code>
$O(\log n)$	Logarithmic	Divide in half	<code>Binary Search</code>
$O(n)$	Linear	1-to-1 ratio	<code>for</code> loop, <code>Array.unshift()</code>
$O(n \log n)$	Linearithmic	Looping a halving	<code>Merge Sort</code>
$O(n^2)$	Quadratic	Grid expansion	<code>Nested for</code> loops
$O(2^n)$	Exponential	Branching double	Recursive multiple calls
$O(n!)$	Factorial	Every permutation	Traveling Salesman

Bookmark this dashboard. Keep your code **scalable**.